AGENTIC AI WORKSHOP · DAY 10 OF 10

The Java bridge & capstone

A Java agent built in Spring AI and LangChain4j uses the SAME MCP server and A2A critic as the Python system.

You build: **Cross-stack** · about 4.6 h of material

Contents

1. Payments runs on the JVM — Why this matters · 5 min
2. Python ↔ Java concept map — Theory · 10 min
3. Spring AI and LangChain4j — Theory · 10 min
4. Lab 1: A Spring AI service — Lab · intermediate · 30 min
5. Lab 2: A LangChain4j agent — Lab · intermediate · 25 min
6. The capstone: two languages, open protocols — Theory · 5 min
7. Capstone: Java agent ↔ MCP ↔ A2A — Lab · advanced · 90 min
8. Guardrails in Java: check before and after the model — Theory · advanced · 8 min
9. Lab 4: Meridian-J with guardrails and unit tests — Lab · advanced · 35 min
10. Spring AI advisors: filters for every model call — Theory · advanced · 8 min
11. Lab 5: Advisors, memory and records you can trust — Lab · advanced · 35 min
12. Java agents in production — Theory · advanced · 5 min
13. Check yourself — Quiz · 5 min
14. Course recap: how Meridian grew — Recap · 5 min
15. Quiz answer key

WHY THIS MATTERS 5 MIN

Payments runs on the JVM

If the protocols are real, the language doesn't matter. Today you prove it.

The systems that actually move money (authorisation, settlement, ledgers) at PSPs, banks and card schemes are mostly **Java and Spring**. Northstar's platform team won't rewrite in Python to adopt agents. With **MCP** (tools and data) and **A2A** (agents), the Python AI team and the Java platform team share capabilities **without sharing a codebase**. In the capstone you'll build a Java agent, *Meridian-J*, that uses the Python team's MCP server and A2A critic unchanged.

By the end of today you can

- Map every Python concept from this course to its Java equivalent
- Build a Spring AI service and a LangChain4j tool-using agent
- Connect a Java agent to the same MCP server (proof point 1)
- Call the same A2A agent card from Java (proof point 2)

THEORY 10 MIN

Python ↔ Java concept map

The slide everyone photographs.

Python (Days 2–9)	Java
`ChatOpenRouter` (LangChain)	Spring AI `ChatClient` / LangChain4j `ChatModel` (OpenAI-compatible, pointed at OpenRouter)
`ChatPromptTemplate`	Spring AI `PromptTemplate`
`.with_structured_output(Model)`	Spring AI `.call().entity(Record.class)`
`@tool`	`@Tool` (Spring AI and LangChain4j)
`create_agent`	LangChain4j **AI Services** (`AiServices`)
checkpointer / memory	LangChain4j `ChatMemory` + `@MemoryId`
MCP client (Day 6)	Spring AI MCP client starter / LangChain4j `McpToolProvider`
A2A client (Day 7)	A2A Java SDK (alpha) or plain JSON-RPC over `RestClient`
Langfuse / OpenTelemetry	Micrometer + OpenTelemetry

Worth knowing: *Embabel*, a Spring-AI-native agent framework that plans with goals and actions (GOAP). Not today.

THEORY 10 MIN

Spring AI and LangChain4j

Spring AI ≈ 'LangChain the Spring way'. LangChain4j ≈ 'agents for Java'. They work fine together.

Spring AI: a fluent `ChatClient`, prompt templates, typed output straight into a Java `record`:

`RiskController.java (Spring AI)`

```java
@RestController
class RiskController {
    private final ChatClient chat;
    RiskController(ChatClient.Builder builder) {
        this.chat = builder.defaultSystem("You are a payments risk analyst at Northstar Pay (fictional PSP).").build();
    }

    record Explain(String term, String audience) {}
    record LimitVerdict(String merchantId, double mtdVolume, double limit, String currency,
                        double utilisationPct, String verdict) {}

    @PostMapping("/explain")
    String explain(@RequestBody Explain req) {
        return chat.prompt()
            .user(u -> u.text("Explain {term} to a {audience} in 3 bullets.")      // PromptTemplate under the hood
                        .param("term", req.term()).param("audience", req.audience()))
            .call().content();
    }

    @PostMapping("/verdict")
    LimitVerdict verdict(@RequestBody String facts) {
```

```
        return chat.prompt().user("Give the processing-limit verdict for: " + facts)
            .call().entity(LimitVerdict.class);                                    // structured output → record
    }
}
```

LangChain4j AI Services: declare an interface and LangChain4j implements the agent loop, with tools and memory.

```
interface SettlementDesk {
    @SystemMessage("You are Meridian-J, Northstar Pay's settlement desk agent. Use tools for every number.")
    String chat(@MemoryId String analystId, @UserMessage String question);
}

SettlementDesk desk = AiServices.builder(SettlementDesk.class)
    .chatModel(model)                                              // 1.x name (was chatLanguageModel)
    .tools(new PaymentsTools())
    .chatMemoryProvider(id -> MessageWindowChatMemory.withMaxMessages(30))
    .build();
```

Both versions here reach Claude through **OpenRouter's OpenAI-compatible endpoint** (`https://openrouter.ai/api/v1`), with the same key as the Python labs.

LAB INTERMEDIATE 30 MIN

Lab 1: A Spring AI service

Goal. A Spring Boot 4 service on the same OpenRouter key, with plain and typed endpoints, plus (stretch) the Python MCP server as tools.

1. Create the project

start.spring.io: Boot **4.1.x**, Java **21+**, Maven, **Spring Web**. Import the BOMs:

pom.xml (BOMs)

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.1</version>
</parent>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>2.0.1</version>
      <type>pom</type><scope>import</scope>
    </dependency>
    <dependency>
      <groupId>dev.langchain4j</groupId>
      <artifactId>langchain4j-bom</artifactId>
      <version>1.21.0</version>
      <type>pom</type><scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

2. Add the model starter

pom.xml

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-openai</artifactId>   <!-- OpenRouter speaks the OpenAI-compatible API -->
</dependency>
```

3. Configure Claude via OpenRouter

application.yml

```
spring:
  ai:
    openai:
      api-key: ${OPENROUTER_API_KEY}
      base-url: https://openrouter.ai/api/v1       # Spring AI 2.x appends /chat/completions (1.x added /v1 itself: a 404 now)
      chat:
        options:
          model: anthropic/claude-opus-5.5         # OpenRouter model ID (Claude Opus 5.5)
          max-tokens: 4096
          # don't set temperature / top-p / top-k: Claude 5.x rejects sampling params.
          # If you see a 400 about temperature, check the starter isn't sending a default (set it to null in a ChatOptions bean).
  threads:
    virtual:
      enabled: true
```

4. Add the controller and run

Use the `RiskController` from the previous step.

terminal

```
./mvnw spring-boot:run
curl -s localhost:8080/explain -H 'content-type: application/json' \
     -d '{"term":"rolling reserve","audience":"new merchant onboarding analyst"}'
curl -s localhost:8080/verdict -H 'content-type: text/plain' \
     -d 'ACME: MTD volume 9.92M USD, monthly processing limit 9.5M USD'
# → {"merchantId":"ACME","mtdVolume":9920000.0,"limit":9500000.0,"currency":"USD","utilisationPct":104.4,"verdict":"OVER_LIMIT"}
```

5. Stretch: Spring AI MCP client → the Python server

`pom.xml`

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-client</artifactId>
</dependency>
```

6. Point it at the MCP server

Property names for auth headers vary between Spring AI releases; check the MCP Client Boot Starter docs.

`application.yml`

```
spring:
  ai:
    mcp:
      client:
        type: SYNC
        streamable-http:
          connections:
            northstar-kb:
              url: http://localhost:8000
              endpoint: /mcp
```

7. Give the MCP tools to the ChatClient

`RiskController.java`

```
RiskController(ChatClient.Builder builder, ToolCallbackProvider mcpTools) {
    this.chat = builder.defaultToolCallbacks(mcpTools).build();
}
```

8. If something goes wrong

Symptom	Fix
404 from OpenRouter	Spring AI 2.x appends only `/chat/completions`: the base URL must end in `/v1` (`https://openrouter.ai/api/v1`)
400 about `temperature`	Claude 5.x rejects sampling parameters: don't set them
Calls go to api.openai.com	`spring.ai.openai.base-url` isn't being read: check the property name and profile
`conversationId cannot be null`	with a memory advisor, Spring AI 2.x needs `ChatMemory.CONVERSATION_ID` on every call

Expected result. /verdict returns `{"merchantId":"ACME",…,"verdict":"OVER_LIMIT"}`; with MCP, the Python search_policies answers '30 days'.

LAB INTERMEDIATE 25 MIN

Lab 2: A LangChain4j agent

Goal. Meridian-J: the Java twin of the Day 2 agent, with the same data, tools and answers.

1. Dependencies

`pom.xml`

```
<dependency><groupId>dev.langchain4j</groupId><artifactId>langchain4j</artifactId></dependency>
<dependency><groupId>dev.langchain4j</groupId><artifactId>langchain4j-open-ai</artifactId></dependency>   <!-- OpenRouter = OpenAI-compatible API -->
```

2. The toy payments tools

The same merchants and FX rates as the Python labs.

`PaymentsTools.java`

```
public class PaymentsTools {

    record Batch(String batchId, String method, double amount, String currency) {}
    record Merchant(String id, String riskTier, double limit, String limitCurrency, List<Batch> mtd) {}

    private static final Map<String, Merchant> MERCHANTS = Map.of(
        "ACME", new Merchant("ACME", "Medium", 9_500_000, "USD", List.of(
            new Batch("B-1001", "card_cnp", 4_200_000, "EUR"),
            new Batch("B-1002", "card_cnp", 3_100_000, "USD"),
            new Batch("B-1003", "wallet",   2_000_000, "EUR"))),
        "BOLT", new Merchant("BOLT", "Low", 5_000_000, "GBP", List.of(
            new Batch("B-2001", "card_cnp", 1_500_000, "USD"),
            new Batch("B-2002", "card_cnp", 2_000_000, "GBP"))),
        "CRUX", new Merchant("CRUX", "High", 800_000_000, "INR", List.of(
            new Batch("B-3001", "upi",      650_000_000, "INR"),
            new Batch("B-3002", "card_cnp", 2_000_000,   "USD"))));

    private static final Map<String, Double> FX = Map.of(
        "EUR/USD", 1.10, "GBP/USD", 1.30, "USD/INR", 85.0);

    @Tool("Look up a merchant: risk tier, monthly processing limit (amount + currency) and month-to-date settlement batches")
    public Merchant lookupMerchant(@P("merchant id, e.g. ACME") String merchantId) {
        var m = MERCHANTS.get(merchantId.toUpperCase());
        if (m == null) throw new IllegalArgumentException("unknown merchant; valid: " + MERCHANTS.keySet());
        return m;
    }

    @Tool("Spot FX rate converting 1 unit of base into quote. Use for ANY currency conversion; never guess rates.")
    public double getFxRate(@P("ISO code you have") String base, @P("ISO code you want") String quote) {
        if (base.equals(quote)) return 1.0;
        Double r = FX.get(base + "/" + quote);
        if (r != null) return r;
        Double inv = FX.get(quote + "/" + base);
        if (inv != null) return 1.0 / inv;
        throw new IllegalArgumentException("no rate for " + base + "/" + quote + "; convert via USD");
    }

    @Tool("Multiply or add numbers exactly. Use for all arithmetic.")
    public double sum(List<Double> values) { return values.stream().mapToDouble(Double::doubleValue).sum(); }

    @Tool("Multiply two numbers exactly.")
    public double multiply(double a, double b) { return a * b; }
}
```

3. The agent (AI Service) and main()

Note `.baseUrl("https://openrouter.ai/api/v1")` : Claude via OpenRouter.

`Main.java`

```
interface MeridianJ {
    @SystemMessage("""
        You are Meridian-J, the settlement desk agent at Northstar Pay (fictional payment processor).
        Use tools for every number. Convert all amounts into the limit currency before comparing.
        Finish with OVER_LIMIT / NEAR_LIMIT (>90%) / OK and the utilisation %.""")
    String ask(@MemoryId String analystId, @UserMessage String question);
}

public class Main {
    public static void main(String[] args) {
```

```
        ChatModel model = OpenAiChatModel.builder()                 // Claude via OpenRouter's OpenAI-compatible API
            .baseUrl("https://openrouter.ai/api/v1")
            .apiKey(System.getenv("OPENROUTER_API_KEY"))
            .modelName("anthropic/claude-opus-5.5")
            .maxTokens(8000)
            // no temperature/topP/topK: rejected on current Claude models
            .logRequests(true).logResponses(true)
            .build();

        MeridianJ agent = AiServices.builder(MeridianJ.class)
            .chatModel(model)
            .tools(new PaymentsTools())
            .chatMemoryProvider(id -> MessageWindowChatMemory.withMaxMessages(30))
            .maxSequentialToolsInvocations(10)              // ≈ Day 2 MAX_TURNS (check name in your version)
            .build();

        System.out.println(agent.ask("analyst-42", "Is ACME over its processing limit?"));
        System.out.println(agent.ask("analyst-42", "And BOLT?"));     // memory: same analyst id
    }
}
```

4. **Run it and compare**

   Turn on request logging and find the tool-use loop in the HTTP traffic: it's the same dance as Day 2.

5. **Using a local model server?**

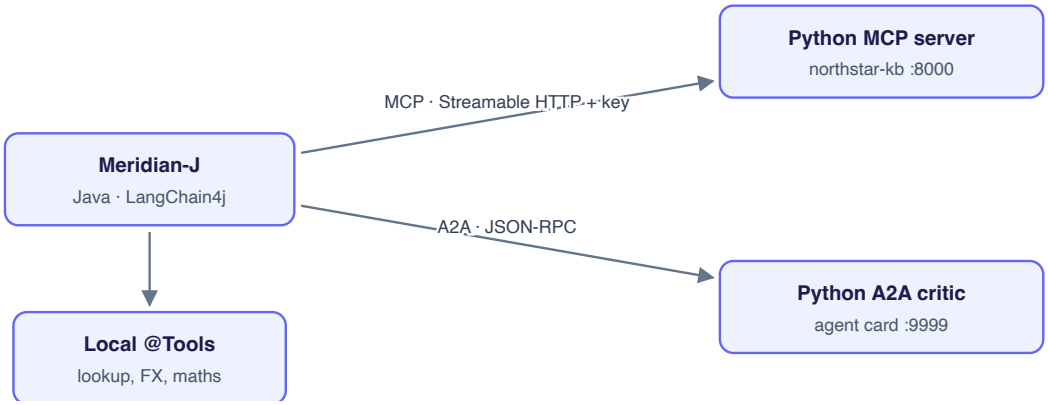
   On a plain `http://` URL (vLLM, Ollama), Java's HTTP client tries an HTTP/2 upgrade and Python servers drop the request body: **400 "body: Field required"**. Force HTTP/1.1 with `.httpClientBuilder(JdkHttpClient.builder().httpClientBuilder(HttpClient.newBuilder().version(HttpClient.Version.HTTP_1_1)))` (Lab 4 shows it).

Expected result. ACME → **OVER_LIMIT** (9.92M USD, 104.4%); 'And BOLT?' works thanks to chat memory → **OK** (63%).

THEORY 5 MIN

The capstone: two languages, open protocols

Meridian-J talks to the Python systems



1. Meridian-J computes each merchant's MTD volume vs limit with local Java tools.
2. Proof point 1: it asks the PYTHON MCP server about policy and eligibility: CRUX isn't eligible (watchlist).
3. Proof point 2: it discovers the PYTHON critic through its agent card and sends the draft for review.
4. Nothing on the Python side changed. The protocol boundary held across a language switch.

LAB ADVANCED 90 MIN

Capstone: Java agent ↔ MCP ↔ A2A

Teams of 2–3. The facilitator runs the Python MCP server and A2A critic on one shared machine.

Goal. Meridian-J prepares the month-end limit summary using its own tools, the Python MCP server and the Python A2A critic.

1. Add the MCP module

pom.xml

```
<dependency><groupId>dev.langchain4j</groupId><artifactId>langchain4j-mcp</artifactId></dependency>   <!-- 1.21.0-beta31 via BOM --
>
```

2. Connect to the same MCP server (with the Day 9 key)

Proof point 1: ask "Can CRUX get a processing limit increase?" and watch the Python server log a call from a Java client.

Main.java: MCP

```
McpTransport transport = StreamableHttpMcpTransport.builder()
    .url("http://localhost:8000/mcp")
    .customHeaders(Map.of("Authorization", "Bearer " + System.getenv("MERIDIAN_KEY")))   // Day 9 auth (check builder method name)
    .logRequests(true).logResponses(true)
    .build();

McpClient northstarKb = DefaultMcpClient.builder()
    .key("northstar-kb")
    .transport(transport)
    .build();

McpToolProvider mcpTools = McpToolProvider.builder()
    .mcpClients(northstarKb)
    .build();

MeridianJ agent = AiServices.builder(MeridianJ.class)
    .chatModel(model)
    .tools(new PaymentsTools(), new CriticA2ATool())          // local tools + A2A tool (Part 2)
    .toolProvider(mcpTools)                                    // search_policies, get_merchant_risk_profile from Python
    .chatMemoryProvider(id -> MessageWindowChatMemory.withMaxMessages(40))
    .build();
```

3. Discover the critic's agent card

terminal

```
curl -s http://127.0.0.1:9999/.well-known/agent-card.json | jq '.name, .skills[].id, .supportedInterfaces'
```

4. Call the critic over A2A (plain JSON-RPC)

Proof point 2. The A2A Java SDK is alpha, so this version uses `RestClient`: the protocol is just HTTP + JSON. Check the method name and message shape against the agent card.

CriticA2ATool.java

```
public class CriticA2ATool {
    private final RestClient http = RestClient.create();
    private final ObjectMapper json = new ObjectMapper();

    @Tool("Send a draft merchant risk summary to Northstar's independent A2A critic agent. Returns its JSON verdict.")
    public String reviewWithCritic(@P("the full draft summary text") String draft) throws Exception {
        // 1) discover
        JsonNode card = json.readTree(http.get().uri("http://127.0.0.1:9999/.well-known/agent-card.json")
                                          .retrieve().body(String.class));
        String url = card.path("supportedInterfaces").get(0).path("url").asText();

        // 2) send a message (A2A JSON-RPC binding). Method name and message shape per A2A protocol v1.0.
        //    Check them against the critic's agent card / a2a-inspector during the dry run.
        Map<String, Object> rpc = Map.of(
            "jsonrpc", "2.0", "id", UUID.randomUUID().toString(), "method", "SendMessage",
            "params", Map.of("message", Map.of(
                "messageId", UUID.randomUUID().toString(),
                "role", "ROLE_USER",
                "parts", List.of(Map.of("text", "Review this merchant risk summary:\n" + draft)))));

        String resp = http.post().uri(url).contentType(MediaType.APPLICATION_JSON)
                          .header("A2A-Version", "1.0")
                          .body(json.writeValueAsString(rpc)).retrieve().body(String.class);
```

```
            return resp;   // the model reads the task status / message containing {"approved":..., "issues":[...]}
        }
    }
```

5. **Run the full scenario**

   "Prepare the month-end merchant limit summary for ACME, BOLT and CRUX, check who can approve increases and whether CRUX is eligible, then have the critic review it."

6. **Demo**

   Show three terminals: the Java console (local + MCP + A2A calls), the Python MCP server log, and the A2A critic log (submitted → working → completed).

Expected result. Correct verdicts for all three merchants, the policy answer from the Python MCP server, and the Python critic's verdict, all from a Java agent.

THEORY ADVANCED 8 MIN

Guardrails in Java: check before and after the model

LangChain4j runs **input guardrails** before a request reaches the model and **output guardrails** on every answer:

	Example	Result types
Input	a card number in the question	`success()`, `failure(..)`, `fatal(..)` (stop: the text never reaches the model or its provider's logs)
Output	the verdict contradicts the percentage	`success()`, `reprompt(reason, instruction)` (ask the model to fix it), `fatal(..)`

`@OutputGuardrails(value = VerdictGuardrail.class, maxRetries = 2)`: **maxRetries counts model calls in total**, so 2 means the first answer plus one reprompt.

Guardrails are plain Java classes, so you **unit-test them with a scripted fake model**: no network, no tokens, exact results.

LAB ADVANCED 35 MIN

Lab 4: Meridian-J with guardrails and unit tests

Goal. A LangChain4j agent that blocks card numbers before the model and re-prompts when its verdict contradicts its numbers, with tests that need no LLM.

1. The project

`langchain4j-http-client-jdk` is listed explicitly because `Main` configures the HTTP client.

`pom.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.northstar</groupId>
  <artifactId>meridian-guardrails</artifactId>
  <version>1.0.0</version>
  <properties>
    <maven.compiler.release>25</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencyManagement>
    <dependencies>
      <dependency><groupId>dev.langchain4j</groupId><artifactId>langchain4j-bom</artifactId><version>1.21.0</version>
<type>pom</type><scope>import</scope></dependency>
      <dependency><groupId>org.junit</groupId><artifactId>junit-bom</artifactId><version>6.1.3</version><type>pom</type>
<scope>import</scope></dependency>
    </dependencies>
  </dependencyManagement>
  <dependencies>
    <dependency><groupId>dev.langchain4j</groupId><artifactId>langchain4j</artifactId></dependency>
    <dependency><groupId>dev.langchain4j</groupId><artifactId>langchain4j-open-ai</artifactId></dependency>
    <dependency><groupId>dev.langchain4j</groupId><artifactId>langchain4j-http-client-jdk</artifactId></dependency>
    <dependency><groupId>org.slf4j</groupId><artifactId>slf4j-simple</artifactId><version>2.0.17</version></dependency>
    <dependency><groupId>org.junit.jupiter</groupId><artifactId>junit-jupiter</artifactId><scope>test</scope></dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-compiler-plugin</artifactId><version>3.16.0</version>
<configuration><parameters>true</parameters></configuration></plugin>
      <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven-surefire-plugin</artifactId><version>3.6.0</version>
</plugin>
      <plugin><groupId>org.codehaus.mojo</groupId><artifactId>exec-maven-plugin</artifactId><version>3.6.2</version><configuration>
<mainClass>com.northstar.meridian.Main</mainClass></configuration></plugin>
    </plugins>
  </build>
</project>
```

2. The tools

`MerchantTools.java`

```
package com.northstar.meridian;

import dev.langchain4j.agent.tool.Tool;
import java.util.Map;

/** Northstar's data as tools the model can call. Same numbers as the Python course. */
public class MerchantTools {
    private static final Map<String, String> MERCHANTS = Map.of(
        "ACME", "limit 9,500,000 USD; batches 4,200,000 EUR, 3,100,000 USD, 2,000,000 EUR; Medium tier",
        "BOLT", "limit 5,000,000 GBP; batches 1,500,000 USD, 2,000,000 GBP; Low tier",
        "CRUX", "limit 800,000,000 INR; batches 650,000,000 INR, 2,000,000 USD; High tier; chargeback watchlist");
    private static final Map<String, Double> FX = Map.of("EUR/USD", 1.10, "GBP/USD", 1.30, "USD/INR", 85.0);

    @Tool("Look up a merchant file: monthly processing limit, month-to-date settlement batches, risk tier")
    public String lookupMerchant(String merchantId) {
        String m = MERCHANTS.get(merchantId.trim().toUpperCase());
        return m != null ? m : "unknown merchant; valid: " + MERCHANTS.keySet();
    }

    @Tool("Spot FX rate: how many units of `quote` one unit of `base` buys. Use for ANY currency conversion.")
    public double fxRate(String base, String quote) {
        String b = base.toUpperCase(), q = quote.toUpperCase();
        if (b.equals(q)) return 1.0;
        Double direct = FX.get(b + "/" + q);
        if (direct != null) return direct;
```

```
        Double inverse = FX.get(q + "/" + b);
        if (inverse != null) return 1 / inverse;
        throw new IllegalArgumentException("no rate for " + b + "/" + q);
    }

    @Tool("Multiply two numbers")
    public double multiply(double a, double b) { return a * b; }

    @Tool("Add a list of numbers")
    public double sum(double[] values) {
        double total = 0;
        for (double v : values) total += v;
        return total;
    }
}
```

3. The input guardrail

13–19 digits that pass the Luhn check: `fatal`, so the request never leaves your JVM.

PanInputGuardrail.java

```
package com.northstar.meridian;

import dev.langchain4j.data.message.UserMessage;
import dev.langchain4j.guardrail.InputGuardrail;
import dev.langchain4j.guardrail.InputGuardrailResult;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/** Blocks a request BEFORE it reaches the model if it contains a card number (13–19 digits passing the Luhn check). */
public class PanInputGuardrail implements InputGuardrail {
    private static final Pattern DIGITS = Pattern.compile("(?:\\d[ -]?){12,18}\\d");

    @Override
    public InputGuardrailResult validate(UserMessage message) {
        Matcher m = DIGITS.matcher(message.singleText());
        while (m.find()) {
            String digits = m.group().replaceAll("[ -]", "");
            if (luhn(digits)) {
                // fatal: stop here, never send this text to the model (or to the model provider's logs)
                return fatal("This message contains a card number. Remove it: Meridian must never see card data.");
            }
        }
        return success();
    }

    static boolean luhn(String digits) {
        int sum = 0;
        boolean dbl = false;
        for (int i = digits.length() - 1; i >= 0; i--) {
            int d = digits.charAt(i) - '0';
            if (dbl && (d *= 2) > 9) d -= 9;
            sum += d;
            dbl = !dbl;
        }
        return sum % 10 == 0;
    }
}
```

4. The output guardrail

The business rule (above 100% is OVER_LIMIT…) checked in code. A contradiction triggers `reprompt` with the rule.

VerdictGuardrail.java

```
package com.northstar.meridian;

import dev.langchain4j.data.message.AiMessage;
import dev.langchain4j.guardrail.OutputGuardrail;
import dev.langchain4j.guardrail.OutputGuardrailResult;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
 * Checks the answer against the business rule, in code: the verdict must match the percentage.
 * If it doesn't, the model is asked to fix it (reprompt), up to maxRetries times.
 */
public class VerdictGuardrail implements OutputGuardrail {
    private static final Pattern LINE = Pattern.compile("VERDICT:\\s*(OVER_LIMIT|NEAR_LIMIT|OK)\\s*\\((\\d+(?:\\.\\d+)?)%\\)");

    @Override
```

```
    public OutputGuardrailResult validate(AiMessage answer) {
        Matcher m = LINE.matcher(answer.text() == null ? "" : answer.text());
        if (!m.find()) {
            return reprompt("No verdict line", "End your answer with exactly one line: VERDICT: <OVER_LIMIT|NEAR_LIMIT|OK>
(<percentage>%)");
        }
        double pct = Double.parseDouble(m.group(2));
        String expected = pct > 100 ? "OVER_LIMIT" : pct > 90 ? "NEAR_LIMIT" : "OK";
        if (!m.group(1).equals(expected)) {
            return reprompt("Verdict " + m.group(1) + " doesn't match " + pct + "%",
                "Your verdict contradicts your percentage. The rule: above 100% is OVER_LIMIT, above 90% NEAR_LIMIT, otherwise OK.
"
                + "Recheck the numbers and answer again.");
        }
        return success();
    }
}
```

5. The agent interface

Meridian.java

```
package com.northstar.meridian;

import dev.langchain4j.service.SystemMessage;
import dev.langchain4j.service.guardrail.InputGuardrails;
import dev.langchain4j.service.guardrail.OutputGuardrails;

/** The agent's contract. LangChain4j implements it: tool calls, the loop, and both guardrails. */
@InputGuardrails(PanInputGuardrail.class)
@OutputGuardrails(value = VerdictGuardrail.class, maxRetries = 2)   // at most 2 model calls in total: answer + 1 reprompt
public interface Meridian {
    @SystemMessage("""
        You are Meridian, a merchant-risk analyst at Northstar Pay. Use the tools for every fact and number.
        Convert every batch into the limit currency with fxRate before adding. Use the tools for arithmetic.
        Finish with exactly one line: VERDICT: <OVER_LIMIT|NEAR_LIMIT|OK> (<utilisation percentage>%)""")
    String ask(String question);
}
```

6. Run the tests first

`ScriptedModel` plays back answers you choose: a contradictory verdict, then a fixed one. Expected: **Tests run: 4, Failures: 0**, in well under a second.

GuardrailTest.java

```
package com.northstar.meridian;

import static org.junit.jupiter.api.Assertions.*;

import dev.langchain4j.data.message.AiMessage;
import dev.langchain4j.data.message.ChatMessage;
import dev.langchain4j.data.message.UserMessage;
import dev.langchain4j.guardrail.InputGuardrailException;
import dev.langchain4j.model.chat.ChatModel;
import dev.langchain4j.model.chat.request.ChatRequest;
import dev.langchain4j.model.chat.response.ChatResponse;
import dev.langchain4j.service.AiServices;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.List;
import org.junit.jupiter.api.Test;

/** No LLM: a scripted fake model plays back answers, so we can test the agent's rules exactly and in milliseconds. */
class GuardrailTest {

    static class ScriptedModel implements ChatModel {
        final Deque<String> replies;
        final List<List<ChatMessage>> seen = new ArrayList<>();
        ScriptedModel(String... replies) { this.replies = new ArrayDeque<>(List.of(replies)); }
        @Override public ChatResponse doChat(ChatRequest request) {
            seen.add(request.messages());
            return ChatResponse.builder().aiMessage(AiMessage.from(replies.removeFirst())).build();
        }
    }

    Meridian meridian(ChatModel model) {
        return AiServices.builder(Meridian.class).chatModel(model).tools(new MerchantTools()).build();
    }

    @Test
```

```
    void wrongVerdictIsRepromptedAndFixed() {
        var model = new ScriptedModel(
            "ACME processed 9.92M USD against 9.5M.\nVERDICT: OK (104.4%)",          // contradicts itself
            "ACME processed 9.92M USD against 9.5M.\nVERDICT: OVER_LIMIT (104.4%)"); // fixed after the reprompt
        String answer = meridian(model).ask("Is ACME over its limit?");
        assertTrue(answer.endsWith("VERDICT: OVER_LIMIT (104.4%)"));
        assertEquals(2, model.seen.size(), "the guardrail asked the model once more");
        String reprompt = ((UserMessage) model.seen.get(1).getLast()).singleText();
        assertTrue(reprompt.contains("contradicts"), reprompt);
    }

    @Test
    void cardNumberNeverReachesTheModel() {
        var model = new ScriptedModel("should never be used");
        var e = assertThrows(InputGuardrailException.class, () -> meridian(model).ask("Card 4111 1111 1111 1111, is ACME over?"));
        assertTrue(e.getMessage().contains("card number"));
        assertEquals(0, model.seen.size());
    }

    @Test
    void ordinaryNumbersAreNotCardNumbers() {
        var model = new ScriptedModel("BOLT is fine.\nVERDICT: OK (63.1%)");
        assertEquals("BOLT is fine.\nVERDICT: OK (63.1%)", meridian(model).ask("Batch B-2001 was 1500000 USD; is BOLT near its
limit?"));
    }

    @Test
    void givesUpAfterMaxRetries() {
        var model = new ScriptedModel("VERDICT: OK (150%)", "VERDICT: OK (150%)", "VERDICT: OK (150%)");
        assertThrows(Exception.class, () -> meridian(model).ask("Is ACME over its limit?"));
        // maxRetries = 2 means at most 2 model calls IN TOTAL: the first answer and one reprompt.
        assertEquals(2, model.seen.size());
    }
}
```

```
RUN  ./mvnw TEST
```

7. Run against a real model

Expected:

```
Q: Is ACME over its processing limit?
… $9,920,000 … against a $9,500,000 monthly limit …
VERDICT: OVER_LIMIT (104.42%)

Q: Is BOLT near its limit?
… 3,153,846.15 GBP, which is 63.08% of the limit …
VERDICT: OK (63.08%)

BLOCKED before the model: … This message contains a card number …
```

Main.java

```
package com.northstar.meridian;

import dev.langchain4j.guardrail.InputGuardrailException;
import dev.langchain4j.http.client.jdk.JdkHttpClient;
import dev.langchain4j.model.openai.OpenAiChatModel;
import dev.langchain4j.service.AiServices;
import java.net.http.HttpClient;

public class Main {
    public static void main(String[] args) {
        // Claude via OpenRouter by default; any OpenAI-compatible endpoint works (set BASE_URL / MODEL).
        var model = OpenAiChatModel.builder()
            .baseUrl(System.getenv().getOrDefault("BASE_URL", "https://openrouter.ai/api/v1"))
            .apiKey(System.getenv().getOrDefault("OPENROUTER_API_KEY", "none"))
            .modelName(System.getenv().getOrDefault("MODEL", "anthropic/claude-opus-5.5"))
            .timeout(java.time.Duration.ofMinutes(3))
            // HTTP/1.1: on plain http:// URLs Java's client tries an HTTP/2 upgrade, and Python model servers (vLLM, uvicorn)
            // then drop the request body ("400 body: Field required"). Harmless for https endpoints like OpenRouter.

.httpClientBuilder(JdkHttpClient.builder().httpClientBuilder(HttpClient.newBuilder().version(HttpClient.Version.HTTP_1_1)))
            .build();
        Meridian meridian = AiServices.builder(Meridian.class).chatModel(model).tools(new MerchantTools()).build();

        for (String q : new String[] {"Is ACME over its processing limit?", "Is BOLT near its limit?"}) {
            System.out.println("Q: " + q + "\n" + meridian.ask(q) + "\n");
        }
        try {
```

```
            meridian.ask("Customer paid with 4111 1111 1111 1111. Is ACME over its limit?");
        } catch (InputGuardrailException e) {
            System.out.println("BLOCKED before the model: " + e.getMessage());
        }
    }
}
```

RUN

```
EXPORT OPENROUTER_API_KEY=...
./mvnw compile EXEC:java
```

8. **Break the rule**

In the test, make the second scripted answer still contradict itself. Check that the service throws after `maxRetries`, and decide what your API should return to a user then (a clear error, not a wrong verdict).

Expected result. 4 passing unit tests with no LLM; against a real model, ACME OVER_LIMIT and BOLT OK with consistent verdict lines, and the card-number question blocked before the model.

THEORY ADVANCED 8 MIN

Spring AI advisors: filters for every model call

An **advisor** wraps every `ChatClient` call, like a servlet filter: code before the model (block, enrich, add memory) and after it (audit, check, transform). Advisors run in order (`getOrder()`), so a policy advisor at `HIGHEST_PRECEDENCE` sees the request before memory is added.

Built-in advisor	Does
`MessageChatMemoryAdvisor`	adds the conversation's history (needs `ChatMemory.CONVERSATION_ID` on every call)
`QuestionAnswerAdvisor` / RAG advisors	retrieve documents and add them to the prompt
`SafeGuardAdvisor`	blocks requests containing listed words
yours (`CallAdvisor`)	policy checks, audit logs, token budgets, PII masking

Two traps found while testing this lab:

- With memory, follow-ups ("And BOLT?") can be answered from the conversation instead of the tools: a confident wrong answer. Tell the model to look up **every** merchant it mentions, in every turn.
- `.entity(LimitVerdict.class)` gives you a typed record, **not a correct one**: we got invented currencies and totals. Check records against figures computed in code, retry once, and escalate if it's still wrong.

LAB ADVANCED 35 MIN

Lab 5: Advisors, memory and records you can trust

Goal. A Spring AI ChatClient with a policy/audit advisor and chat memory, whose typed records are checked in code before anyone sees them.

1. The project and config

`pom.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
  </parent>
  <groupId>com.northstar</groupId>
  <artifactId>meridian-advisors</artifactId>
  <version>1.0.0</version>
  <properties>
    <java.version>25</java.version>
  </properties>
  <dependencyManagement>
    <dependencies>
      <dependency><groupId>org.springframework.ai</groupId><artifactId>spring-ai-bom</artifactId><version>2.0.1</version>
<type>pom</type><scope>import</scope></dependency>
    </dependencies>
  </dependencyManagement>
  <dependencies>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter</artifactId></dependency>
    <dependency><groupId>org.springframework.ai</groupId><artifactId>spring-ai-starter-model-openai</artifactId></dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</artifactId></plugin>
    </plugins>
  </build>
</project>
```

2. application.yml

Note the base URL ends in **/v1**: Spring AI 2.x.

`src/main/resources/application.yml`

```
spring:
  main:
    web-application-type: none          # a command-line demo: no web server
  ai:
    openai:
      api-key: ${OPENROUTER_API_KEY:none}
      base-url: ${BASE_URL:https://openrouter.ai/api/v1}   # Spring AI 2.x appends /chat/completions (1.x appended /v1/...)
      chat:
        options:
          model: ${MODEL:anthropic/claude-opus-5.5}
          max-tokens: 4096
  threads:
    virtual:
      enabled: true
logging:
  level:
    root: warn
    com.northstar: info
```

3. Tools, and the figures that matter computed in code

`expected(...)` computes volume, limit, currency and verdict deterministically from the same data the tools return. The model explains; code decides the numbers.

`MerchantTools.java`

```
package com.northstar.meridian;

import java.util.Map;
import org.springframework.ai.tool.annotation.Tool;
```

```
/** Northstar's data as Spring AI tools. */
class MerchantTools {
    private static final Map<String, String> MERCHANTS = Map.of(
        "ACME", "limit 9,500,000 USD; batches 4,200,000 EUR, 3,100,000 USD, 2,000,000 EUR; Medium tier",
        "BOLT", "limit 5,000,000 GBP; batches 1,500,000 USD, 2,000,000 GBP; Low tier",
        "CRUX", "limit 800,000,000 INR; batches 650,000,000 INR, 2,000,000 USD; High tier; chargeback watchlist");
    private static final Map<String, Double> FX = Map.of("EUR/USD", 1.10, "GBP/USD", 1.30, "USD/INR", 85.0);

    /** The figures that matter, computed in code from the same data the tools return. Used to CHECK the model. */
    record Expected(double volume, double limit, String currency, String verdict) {}

    private static final Map<String, Object[]> FILES = Map.of(       // limit, currency, batches (amount, currency)...
        "ACME", new Object[] {9_500_000.0, "USD", 4_200_000.0, "EUR", 3_100_000.0, "USD", 2_000_000.0, "EUR"},
        "BOLT", new Object[] {5_000_000.0, "GBP", 1_500_000.0, "USD", 2_000_000.0, "GBP"},
        "CRUX", new Object[] {800_000_000.0, "INR", 650_000_000.0, "INR", 2_000_000.0, "USD"});

    static Expected expected(String merchantId) {
        Object[] f = FILES.get(merchantId);
        double limit = (double) f[0];
        String ccy = (String) f[1];
        double volume = 0;
        var tools = new MerchantTools();
        for (int i = 2; i < f.length; i += 2) volume += (double) f[i] * tools.fxRate((String) f[i + 1], ccy);
        double pct = volume / limit * 100;
        return new Expected(volume, limit, ccy, pct > 100 ? "OVER_LIMIT" : pct > 90 ? "NEAR_LIMIT" : "OK");
    }

    @Tool(description = "Look up a merchant file: monthly processing limit, month-to-date settlement batches, risk tier")
    String lookupMerchant(String merchantId) {
        String m = MERCHANTS.get(merchantId.trim().toUpperCase());
        return m != null ? m : "unknown merchant; valid: " + MERCHANTS.keySet();
    }

    @Tool(description = "Spot FX rate: how many units of quote one unit of base buys. Use for ANY currency conversion.")
    double fxRate(String base, String quote) {
        String b = base.toUpperCase(), q = quote.toUpperCase();
        if (b.equals(q)) return 1.0;
        if (FX.containsKey(b + "/" + q)) return FX.get(b + "/" + q);
        if (FX.containsKey(q + "/" + b)) return 1 / FX.get(q + "/" + b);
        throw new IllegalArgumentException("no rate for " + b + "/" + q);
    }
}
```

4. The policy and audit advisor

PolicyAdvisor.java

```
package com.northstar.meridian;

import java.util.regex.Pattern;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.ai.chat.client.ChatClientRequest;
import org.springframework.ai.chat.client.ChatClientResponse;
import org.springframework.ai.chat.client.advisor.api.CallAdvisor;
import org.springframework.ai.chat.client.advisor.api.CallAdvisorChain;
import org.springframework.core.Ordered;

/**
 * An advisor wraps every call, like a servlet filter: code BEFORE the model (block bad requests) and AFTER it
 * (audit: who asked, how long it took, how many tokens). Every ChatClient call goes through it.
 */
class PolicyAdvisor implements CallAdvisor {
    private static final Logger log = LoggerFactory.getLogger(PolicyAdvisor.class);
    private static final Pattern CARD = Pattern.compile("\\b(?:\\d[ -]?){12,18}\\d\\b");

    static class PolicyViolation extends RuntimeException {
        PolicyViolation(String message) { super(message); }
    }

    @Override
    public ChatClientResponse adviseCall(ChatClientRequest request, CallAdvisorChain chain) {
        String text = request.prompt().getUserMessage().getText();
        if (CARD.matcher(text).find()) {
            throw new PolicyViolation("blocked: the request contains what looks like a card number");   // never reaches the model
        }
        long start = System.nanoTime();
        ChatClientResponse response = chain.nextCall(request);                                          // the model (and tools)
        var usage = response.chatResponse().getMetadata().getUsage();
        log.info("AUDIT conversation={} ms={} tokens={}", request.context().get("chat_memory_conversation_id"),
                 (System.nanoTime() - start) / 1_000_000, usage.getTotalTokens());
        return response;
```

```
    }

    @Override public String getName() { return "PolicyAdvisor"; }
    @Override public int getOrder() { return Ordered.HIGHEST_PRECEDENCE; }    // outermost: runs before memory is added
}
```

5. The app

`AdvisorsApp.java`

```
package com.northstar.meridian;

import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.client.advisor.MessageChatMemoryAdvisor;
import org.springframework.ai.chat.memory.ChatMemory;
import org.springframework.ai.chat.memory.MessageWindowChatMemory;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class AdvisorsApp {
    record LimitVerdict(String merchantId, double mtdVolume, double limit, String currency, double utilisationPct, String verdict)
{}

    public static void main(String[] args) { SpringApplication.run(AdvisorsApp.class, args); }

    /** Which conversation's memory to use. With a memory advisor configured, Spring AI 2.x requires one on EVERY call. */
    static java.util.function.Consumer<ChatClient.AdvisorSpec> memoryOf(String conversationId) {
        return a -> a.param(ChatMemory.CONVERSATION_ID, conversationId);
    }

    /** Deterministic checks the model can't argue with. null = the record is right. */
    static String check(LimitVerdict v, MerchantTools.Expected e) {
        if (!e.currency().equals(v.currency())) return "the limit currency is " + e.currency() + ", not " + v.currency();
        if (Math.abs(v.mtdVolume() - e.volume()) > e.volume() * 0.005) return "MTD volume should be " + Math.round(e.volume()) + "
" + e.currency();
        if (Math.abs(v.limit() - e.limit()) > 0.5) return "the limit is " + Math.round(e.limit());
        return e.verdict().equals(v.verdict()) ? null : "the verdict should be " + e.verdict();
    }

    @Bean
    CommandLineRunner demo(ChatClient.Builder builder) {
        ChatMemory memory = MessageWindowChatMemory.builder().maxMessages(20).build();
        ChatClient meridian = builder
            .defaultSystem("You are Meridian, a merchant-risk analyst at Northstar Pay. Use the tools for every fact and number; "
                           + "convert batches into the limit currency before adding. Verdict: >100% OVER_LIMIT, >90% NEAR_LIMIT,
else OK. "
                           // Memory makes follow-ups ("And BOLT?") feel answerable from the conversation. They aren't:
                           + "For EVERY merchant you mention, call lookupMerchant in this turn, even in follow-up questions. "
                           + "Never reuse or guess another merchant's figures.")
            .defaultTools(new MerchantTools())
            .defaultAdvisors(new PolicyAdvisor(), MessageChatMemoryAdvisor.builder(memory).build())
            .build();

        return args -> {
            // 1. Memory: the follow-up only makes sense if the conversation is remembered.
            for (String q : new String[] {"Is ACME over its processing limit? One line.", "And BOLT? One line."}) {
                String answer = meridian.prompt().user(q)
                    .advisors(memoryOf("analyst-42"))
                    .call().content();
                System.out.println("Q: " + q + "\nA: " + answer + "\n");
            }
            // 2. Structured output: a typed record instead of text. A record can still be WRONG (we saw 830M and "CNY"),
            //    so check it in code against figures computed deterministically, and retry once with the specific problem.
            String ask = "Give the processing-limit verdict for CRUX. Call lookupMerchant and fxRate first.";
            LimitVerdict crux = meridian.prompt().user(ask).advisors(memoryOf("analyst-42")).call().entity(LimitVerdict.class);
            String problem = check(crux, MerchantTools.expected("CRUX"));
            if (problem != null) {
                System.out.println("Record rejected (" + problem + "), asking again");
                crux = meridian.prompt().user(ask + " Your previous answer was wrong: " + problem).advisors(memoryOf("analyst-
42")).call().entity(LimitVerdict.class);
                problem = check(crux, MerchantTools.expected("CRUX"));
            }
            System.out.println("CRUX as a record: " + crux + (problem == null ? "  ✓ checked" : "  ✗ still wrong (" + problem + "):
escalate to a person"));
            System.out.println();
            // 3. The policy advisor stops a request before it reaches the model.
            try {
                meridian.prompt().user("Refund the customer on card 4111 1111 1111 1111").advisors(memoryOf("analyst-
42")).call().content();
            } catch (PolicyAdvisor.PolicyViolation e) {
```

```
                System.out.println("BLOCKED: " + e.getMessage());
            }
        };
    }
}
```

RUN `./mvnw spring-boot:run`

6. **Read the output**

   Expected (your wording will differ):

```
AUDIT conversation=analyst-42 ms=… tokens=…
Q: Is ACME over its processing limit? One line.   → OVER_LIMIT, 104.4% ($9.92M vs $9.5M)
Q: And BOLT? One line.                            → OK, 63.1% (£3.15M vs £5M)     ← needs memory
CRUX as a record: LimitVerdict[…, mtdVolume=8.2E8, limit=8.0E8, currency=INR, utilisationPct=102.5, verdict=OVER_LIMIT]  ✓ checked
BLOCKED: blocked: the request contains what looks like a card number
```

   In 3 runs on the DGX test model, CRUX's record was right twice; once the model invented an AUD record twice, and the check **rejected it and escalated** instead of showing it. With Claude you'll see far fewer retries, but keep the check: it's what makes a wrong record impossible to ship.

7. **Remove the lookup rule**

   Delete the 'For EVERY merchant… call lookupMerchant' sentence from the system prompt and run a few times. Watch for a confident, invented BOLT answer: that's memory without discipline.

8. **Stretch: a token budget advisor**

   Write a `BudgetAdvisor` that reads `usage.getTotalTokens()` after each call, adds it to a per-conversation counter, and throws when a conversation passes 20,000 tokens.

Expected result. Audited calls, a correct memory follow-up for BOLT, a CRUX record that is either verified or rejected (never shown wrong), and the card-number request blocked.

THEORY ADVANCED 5 MIN

Java agents in production

Concern	In the Spring / LangChain4j world
Observability	Spring AI emits Micrometer metrics and traces for model, tool and advisor calls (`gen_ai.*` attributes): export to OpenTelemetry like Day 8
Concurrency	virtual threads (`spring.threads.virtual.enabled=true`): thousands of waiting model calls without a big thread pool
Resilience	timeouts on every client; retries only for transient errors (429, 5xx); circuit breakers (Resilience4j) around flaky tools
Testing	unit-test guardrails, advisors and tools with scripted fake models; contract-test MCP and A2A endpoints; evals in CI (Day 8)
Config	model, base URL and keys from the environment or a secret store; never in `application.yml` committed to git
Same protocols	MCP and A2A clients in Java talk to the Python servers unchanged: that's the point of the capstone

QUIZ 5 MIN

Check yourself

1. **Java equivalent of create_agent?**
   A. Spring Boot
   B. LangChain4j AI Services
   C. Maven
   D. RestClient
2. **Java equivalent of with_structured_output(Model)?**
   A. A JSON string
   B. .call().entity(Record.class) in Spring AI
   C. Jackson only
   D. There isn't one
3. **What did the Python team change to support the Java agent?**
   A. Rewrote the server
   B. Nothing
   C. Added a Java SDK
   D. Changed the model
4. **Which LangChain4j class turns an MCP server into tools?**
   A. ChatMemory
   B. McpToolProvider
   C. AiServices
   D. RestClient
5. **How does Meridian-J learn what the critic can do?**
   A. From the source code
   B. It fetches the agent card
   C. From a wiki
   D. It guesses
6. **Spring AI 2.x returns 404 from OpenRouter. Most likely cause?**
   A. Wrong key
   B. The base URL doesn't end in /v1 (2.x appends only /chat/completions)
   C. The model is down
   D. Missing temperature
7. **@OutputGuardrails(maxRetries = 2) allows how many model calls in total?**
   A. 1
   B. 2: the first answer plus one reprompt
   C. 3
   D. Unlimited
8. **call().entity(LimitVerdict.class) returned a record. Is it right?**
   A. Yes, it's typed
   B. Not necessarily: check it against figures computed in code, retry once, escalate if still wrong
   C. Only if it parses
   D. Only with streaming
9. **Why does chat memory need extra care with tools?**
   A. It's slow
   B. Follow-ups can be answered from the conversation instead of the tools, confidently and wrongly
   C. It leaks keys
   D. It disables tools

RECAP 5 MIN

Course recap: how Meridian grew

- Java agents get the same discipline: guardrails before and after the model, advisors around every call, and typed results checked in code.
- Day 1–2: an agent is a loop you own: tools, planning, a critic, guardrails.
- Day 3–4: frameworks and graphs: structured output, routing, memory, human approval.
- Day 5–6: knowledge that checks itself (agentic RAG), shared through MCP.

- Day 7–8: crews and A2A, measured with evals, traces and cost ledgers.
- Day 9–10: secured and deployed, and proven across languages through open protocols.

Next: Pick one pattern from this course and apply it at work next week: an approval gate, an eval set, or a route-by-uncertainty split.

Quiz answer key

1. **B. LangChain4j AI Services**
   Declare an interface; LangChain4j implements the agent loop.
2. **B. .call().entity(Record.class) in Spring AI**
   Typed records instead of pydantic models.
3. **B. Nothing**
   That's the proof: the protocols are the contract.
4. **B. McpToolProvider**
   With DefaultMcpClient and StreamableHttpMcpTransport.
5. **B. It fetches the agent card**
   A2A discovery at /.well-known/agent-card.json.
6. **B. The base URL doesn't end in /v1 (2.x appends only /chat/completions)**
   Spring AI 1.x added /v1 itself; 2.x uses the official OpenAI SDK, which doesn't.
7. **B. 2: the first answer plus one reprompt**
   We confirmed it with a scripted fake model: 2 calls, then the service throws.
8. **B. Not necessarily: check it against figures computed in code, retry once, escalate if still wrong**
   Typed ≠ correct: we saw invented currencies and totals.
9. **B. Follow-ups can be answered from the conversation instead of the tools, confidently and wrongly**
   Require a tool lookup for every entity mentioned, in every turn.